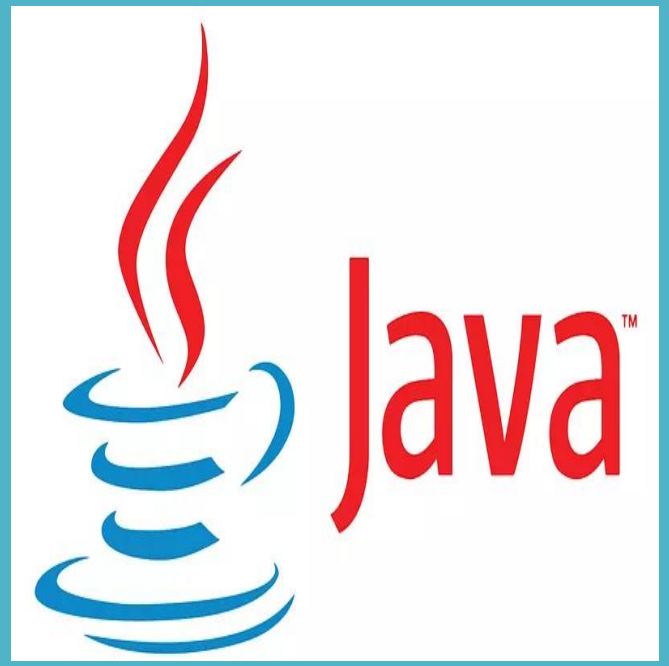
UNIVERSITÉ SAINT JEAN
SAINT JEAN INGÉNIEUR



JAVA WEB PROGRAMMING ISI 3196

LECTURER
Benoît Lionel MBÉLÉ

2nd Semester 2025/2026

CHAPTER 4

INTRODUCTION TO SPRING BOOT AND REST APIs

AGENDA

1- RESTful Web Service Fundamentals

2- Introduction to Spring Boot

3- Features of Spring Boot

4- Architecture of Spring Boot

5- What is Maven

6- First Lab with Spring Boot

1- RESTful Web Service Fundamentals

1- RESTful Web Service Fundamentals

Introducing REST APIs

- An API is the means by which a piece of code communicates with another piece of code.
- You might have already written an API for your code or used one in your programs; for example, in Java libraries for collection, input/output, or streams that provide a variety of APIs to perform specific tasks.
- Historically, there were different ways to integrate one application with another – RPC, **Simple Object Access Protocol** (**SOAP**)-based services, and more.

1- RESTful Web Service Fundamentals

Introducing REST APIs

- The integration of apps has become an integral part of software architectures, especially after the boom of the cloud and mobile phones.
- You now have ***social logins***, such as Facebook, Google, and GitHub, which means you can develop your application even without writing an independent login module and get around security issues such as storing passwords in a secure way.
- **These social logins provide APIs using REST and GraphQL. Currently, REST is the most widely used, and it has become a standard for writing APIs for integration and web app consumption.**

1- RESTful Web Service Fundamentals

What is an API?

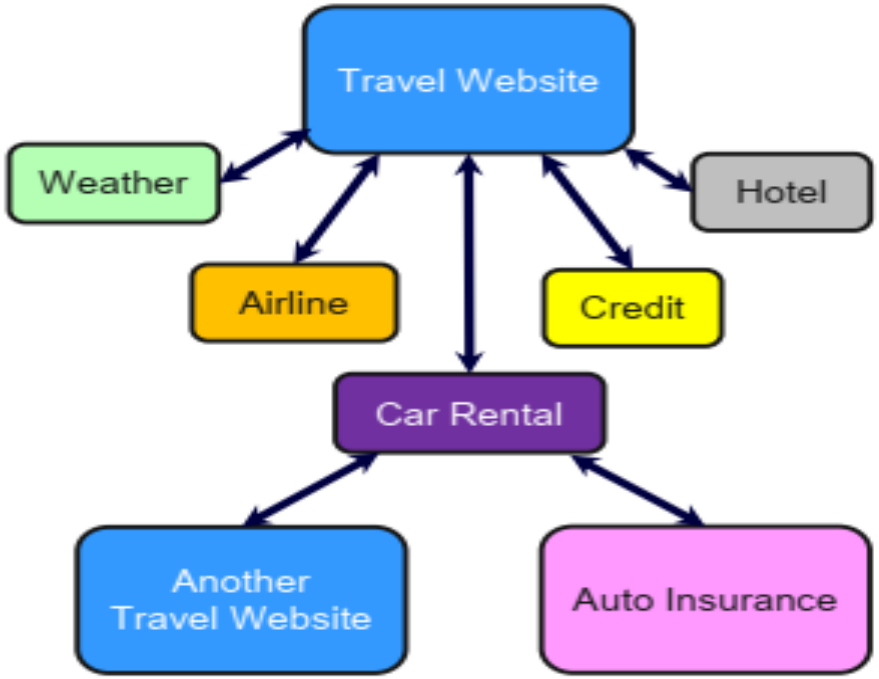
- APIs (Application Program Interfaces) allow applications to communicate with one another
  - *Applications that communicate via APIs can be located on the same computer, over a local network, or over the internet*
- An API is a contract between a client application and a service application
  - *The client application sends a request in an agreed upon format to the API of the service application*
  - *The service application API sends a response back to the client in an agreed upon format*
  - *Neither the client application nor the service application need to know the implementation details of the other*
- APIs allow access to resources while maintaining security and control

1- RESTful Web Service Fundamentals

APIs in Action

Consider a travel website

- ***Its "product" is a one-stop shop for a customer's travel needs***
- ***The travel company itself does not directly provide nor control the travel services***
- ***It must rely on other companies for these services, needs access to their data, schedules, etc, and does so via APIs***
  - Weather – warn customers of advisories and warnings
  - Airline – compare fares and schedules, book flights
  - Car Rental – compare rates and availability, reserve cars
  - Hotel – compare rates and availability, reserve rooms
  - Credit – payments



Now consider the benefits the service provider gains by making an API available

- ***By integrating its services into the travel website, it increases the opportunity for sales***
- ***With little or no extra development, it can expand its presence***
  - Other travel websites
  - Other industries
    - For example, a Car Rental company can make its services and data available to an auto insurance company for clients whose car has been in an accident

1- RESTful Web Service Fundamentals

Introducing REST APIs

- REST stands for **REpresentational State Transfer**, which is a style of software architecture.
- Web services that adhere to the REST style are called RESTful web services. You might have already written an API for your code or used one in your programs; for example, in Java libraries for collection, input/output, or streams that provide a variety of APIs to perform specific tasks.
- Roy Fielding, in his doctoral research, ***Architectural Styles and the Design of Network-Based Software Architectures* (https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm),** came up with REST in 2000.
- REST's architecture style allowed any server to communicate with any other server over the network.
- It simplified communication and made integration easier. REST was made to work on top of HTTP, which enables it to be used all over the web and in internal networks.

1- RESTful Web Service Fundamentals

REST fundamentals

- Therefore, REST is based on the **Client-Server** concept. The client calls the REST API and the server responds with a response.
- REST allows a client (that is, a program, web service, or UI app) to talk to a remotely (or locally) running server (or web service) **using HTTP requests and responses.**
- The client sends to the web service with an API command wrapped in an HTTP request to the web.
- This HTTP request may contain a payload (or input) in the form of query parameters, headers, or request bodies. The called web service responds with a success/failure indicator and the response data wrapped inside the HTTP response.
- The HTTP status code normally denotes the status, and the response body contains the response data. For example, an HTTP status code of 200 OK normally represents success.

1- RESTful Web Service Fundamentals

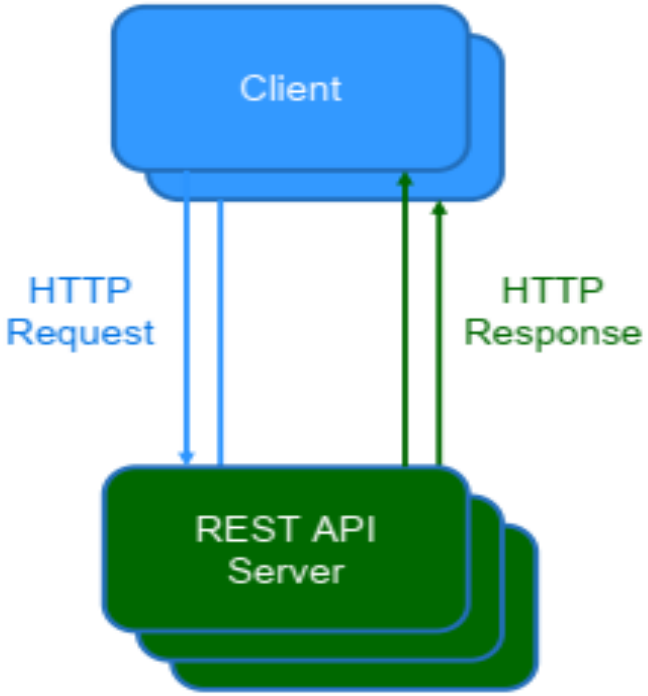
REST fundamentals

- From a REST perspective, an HTTP request is self-descriptive and has enough context for the server to process it. Therefore, REST calls are **stateless**.
- States are either managed on the client side or on the server side. A REST API does not maintain its state.
- It only transfers states from the server to the client or vice versa. Therefore, it is called **REpresentation State Transfer, or REST** for short.

1- RESTful Web Service Fundamentals

Why REST?

- Maintains separation between client and server
  - ***The same interface can be used whether the client is a user interface or another REST API server***
- Provides a uniform interface to access and manage resources
- Scalability and Reliability
  - ***REST APIs can be deployed to multiple servers in different locations***
  - ***If one server becomes unavailable, requests can be automatically routed to another with no loss in service (load balancing)***
  - ***As request volumes increase, additional REST API servers can be brought online***
- Language and Platform Independence
  - ***REST APIs can be written in nearly any language and clients can be written in a completely different language***
  - ***REST APIs can be hosted on nearly any Operating System***
- Flexible Data Formats
  - ***REST APIs can accept and return multiple data formats, e.g. JSON, XML***



1- RESTful Web Service Fundamentals

REST fundamentals

- The following is a list of key concepts in REST:
  - Resources and URIs
  - HTTP methods

1- RESTful Web Service Fundamentals

Resources and URIs

- A resource is identified by a Uniform Resource Identifier
- A URI looks very similar to a website address
- The basic format is

scheme :// host:port / path to resource ? query parameter

- Example

http :// www.state.edu:9150 / se/faculty ? id=310

1- RESTful Web Service Fundamentals

REST HTTP Methods

- The most commonly used HTTP methods in REST carry out CRUD operations (Create, Read, Update, Delete)
  - ***POST – Create a new resource***
    - `POST /petstore/pets/dog` — Create a new dog
  - ***GET – Read access to a resource***
    - `GET /petstore/pets` — Get all pets
    - `GET /petstore/pets/dog/{id}` — Get a specific dog
  - ***PUT – Update or create a resource***
    - `PUT /petstore/pets/dog/{id}` — Update a specific dog
  - ***DELETE – Delete a resource***
    - `DELETE /petstore/pets/dog/{id}` — Delete a specific dog (because it went to a loving home)
- When you type a URL into a browser, an HTTP GET request is sent to the website and the response data is used to render the page

1- RESTful Web Service Fundamentals

REST Request Components

- A REST API request consists of four main parts:
  - ***Method***
    - Required
    - Identifies the operation
    - Example: GET
  - ***URI***
    - Required
    - Identifies the resource
    - May include query parameters to identify specific content or actions
    - Example: `http://localhost:8080/jedi/5`
  - ***Headers***
    - Optional, but generally used
    - Provides additional information about the request or client
    - Applications, e.g. Browsers, or frameworks, e.g. Spring, often add standard and their own headers to requests
      - For the purposes of the term project, we will focus on `Content-Type` and custom headers
    - Example: `Content-Type: application/json`
  - ***Body***
    - Generally used for POST and PUT, but not for GET and DELETE
    - Representation of one or more objects
    - Example: `{"id": 3, "lastName": "Skywalker", "firstName": "Anakin"}`

1- RESTful Web Service Fundamentals

REST Response Components

- A REST API response consists of three main parts:
  - ***Status Code***
    - Required
    - Identifies the result of the operation
    - Example: 200/OK
  - ***Headers***
    - Optional
    - Provides additional information about the response to the client
    - Example: `api-status-code: 3`
  - ***Body***
    - Required for GET, but often used for other methods
    - Representation of one or more objects
    - Example: `{"id": 3, "lastName": "Skywalker", "firstName": "Anakin"}`
- Common HTTP Status Codes
  - `200/OK` – ***Request was completed successfully***
  - `201/Created` – ***Resource was created successfully***
  - `400/Bad Request` – ***Body of request was invalid***
  - `403/Forbidden` – ***Caller does not have permissions for the requested resource***
  - `404/Not Found` – ***Requested resource could not be found***
  - `500/Internal Server Error` – ***Server cannot fulfill request and does not want to expose specifics to client***
  - `501/Not Implemented` – ***Requested method is not currently supported***

1- RESTful Web Service Fundamentals

REST HTTP Methods - POST

- Creates a new resource
- Request
  - ***URI specifies the resource to be created***

    http://localhost:8080/jedi
  - ***Header tells the REST API the format of the Body***

    Content-Type: application/json
  - ***Body is a representation of the jedi object***

```
{
    "lastName": "Skywalker",
    "firstName": "Anakin"
}
```

Notice the "id" field is not included - The unique identifier of a resource should be created and managed by the REST API service unless a field is determined to be unique

- Response
  - ***Common Status Codes***

    201 – CREATED

    403 – FORBIDDEN
  - ***Header***

    Application dependent
  - ***Body is a representation of the created object***

```
{
    "id": 3,
    "lastName": "Skywalker",
    "firstName": "Anakin"
}
```

1- RESTful Web Service Fundamentals

REST HTTP Methods – GET

- Retrieves a resource
- Request
  - *URI provides enough information identify the resource*
    http://localhost:8080/jedi/3
  - *Header*
    Generally not applicable
  - *Body*
    Generally not applicable
- Response
  - *Common Status Codes*
    200 – OK
    404 – NOT FOUND
  - *Header*
    Application dependent
  - *Body is a representation of the object requested*

```
{
   "id": 3,
   "lastName": "Skywalker",
   "firstName": "Anakin"
}
```

If multiple objects are requested, an array would be returned

1- RESTful Web Service Fundamentals

REST HTTP Methods - PUT

- Update a resource or create the resource if it does not exist
- Request
  - *URI provides enough information identify the resource*

    http://localhost:8080/jedi
  - *Header tells the REST API the format of the Body*

    Content-Type: application/json
  - *Body of the request contains an object with the fields to be updated*

```
{
   "id": 3,
   "lastName": "Vader",
   "firstName": "Darth"
}
```

- Response
  - *Common Status Codes*

    200 - OK

    404 - NOT FOUND
  - *Header*

    Application dependent
  - *Body is a representation of the updated object*

```
{
   "id": 3,
   "lastName": "Vader",
   "firstName": "Darth"
}
```

1- RESTful Web Service Fundamentals

REST HTTP Methods – DELETE

- Deletes a resource
- Request
  - *The URI specifies the resource to be deleted*
    http://localhost:8080/jedi/3
  - *Header*
    Generally not applicable
  - *Body*
    Not Applicable
- Response
  - *Common Status Codes*
    - 200 – OK
    - 404 – NOT FOUND
  - *Header*
    Application dependent
  - *Body*
    Not applicable

1- What is REST ?

RESTfull API why ?

- **Rapid Development:** Spring Boot accelerates the development process with pre-configured setups and reduced boilerplate code.
- **RESTful Endpoints:** Spring MVC provides a robust framework for creating and managing RESTful web services.
- **Layered Architecture:** Clear separation of concerns with distinct layers for controller, service, and repository, improving maintainability.
- **Database Integration:** Seamless integration with databases like MySQL using Spring Data JPA for efficient data handling.
- **Flexible Configuration:** Customize application settings through application.properties or application.yml files.

1- What is REST ?

RESTfull API why ?

- **CRUD Operations:** Supports full Create, Read, Update, and Delete operations, making it easy to manage resources.
- **Exception Handling:** Built-in support for handling errors and exceptions in a standardized way.
- **Testing and Validation:** Use Postman or similar tools to test and validate API endpoints.
- **Security:** Integrate with Spring Security for robust authentication and authorization mechanisms.
- **Scalability:** Designed to be easily scalable, supporting both small and large-scale applications.

1- RESTful Web Service Fundamentals

Serialization and Deserialization

- As we've seen, JSON is a human-readable text format
- In our REST API application, we do not want to deal with text, but rather Java objects
  - ***From the previous HTTP examples, you can envision Jedi being a class with 3 fields:***
    - `id – Number`
    - `firstname – String`
    - `lastname – String`
- Serialization is the process of converting an application object (e.g. Java object) to text (or byte stream)
- Deserialization is the reverse – converting text (or byte stream) into an application object
- HTTP POST and PUT *requests* contain a JSON Object (text representation) that needs to be converted to an application object our REST API application code can work with
- Conversely, GET *responses* from our REST API need to be converted from an application object into a JSON object that can be transmitted back to the client
- Additionally, within a REST API service, we usually want to persist data, whether it be in a file, database, or other storage
  - ***As information is typically represented in files and database as text, serialization and deserialization can be used to transform application objects to JSON objects and vice-versa***
  - ***The JSON objects, which are text, are then easily written to and read from a file, a database, etc***

1- RESTful Web Service Fundamentals

JSON

- JavaScript Object Notation – a human readable data interchange format for defining and transmitting objects
- The syntax supports name-value pairs and arrays
  - *A name is surrounded by double quotes and separated from the value by a colon*
  - *A value can a number, string, boolean, array, object, or null*
  - *An array is surrounded by square brackets*
- Curly braces wrap objects
- Commas separate name-value pairs and array elements
- Names follow the camel case convention

1- RESTful Web Service Fundamentals

JSON Example: Student

```
{
    "firstName": "Pete",
    "lastName": "Jones"
    "year": 2,
    "address": {
        "street": "50 Main St",
        "city": "Rochester",
        "state: "NY",
        "zipCode": "14623"
    },
    "classes": ["SWEN-261", "MATH-181", "ENGL-150"],
    "avatar": null,
    "enrolled": true
}
```

Name

Value

String

Number

Object

Array

Null

Boolean

1- RESTful Web Service Fundamentals

REST API Frameworks

- Nearly every language has REST frameworks available, most are open source, that support rapid and reliable development
- We will use Java and the Spring Boot framework in our term project
  - ***Spring Boot provides the scaffolding for stand-alone, light-weight, production-grade REST API applications***
  - ***Includes an embedded Tomcat server that hosts your APIs and makes them available to clients on a network***
  - ***Routes HTTP requests to your class methods for handling***
  - ***Built-in support for serialization and deserialization***
  - ***The Spring Initializr wizard, available at start.spring.io or via VSCode extension, quickly builds a baseline project***
    - You will not need to use Spring Initializr as the starter projects are provided
  - ***Many annotations, e.g. `@RestController`, are available to easily control configuration***
  - ***See the course resources page for more information and helpful links***

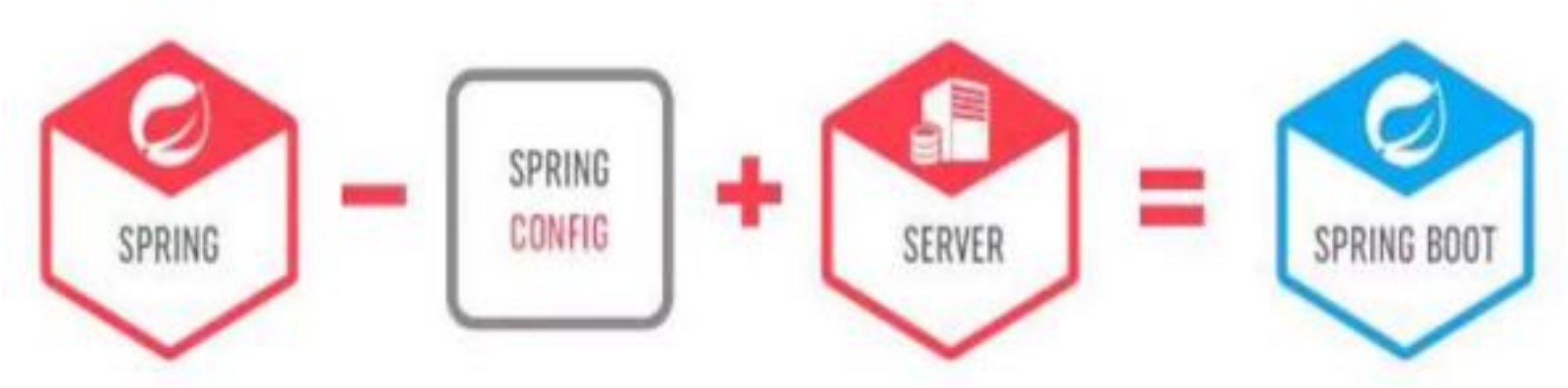
2- Introduction to SPRING BOOT

2- Introduction to Spring Boot

Difference between Spring and Spring Boot

2- Introduction to Spring Boot

SPRING BOOT = code with 0% of configuration!



2- Introduction to Spring Boot

Spring Boot makes it easy to create stand-alone, production-grade Spring based Applications that you can "just run". We take an opinionated view of the Spring platform and third-party libraries so you can get started with minimum fuss. Most Spring Boot applications need very little Spring configuration.

You can use Spring Boot to create Java applications that can be started using java -jar or more traditional war deployments. We also provide a command line tool that runs "spring scripts".

2- Introduction to Spring Boot

WHY We Need Spring Boot?

- Spring Boot is next generation attempt to easy spring setup.
- Spring Boot's main benefit is configuring the resources based on what it finds in the classpath.
- If your Maven POM includes JPA dependencies and a MYSQL driver, then Spring Boot will setup a persistence unit based on MySQL. If you've added a web dependency, then you will get Spring MVC configured with defaults.
- When we talk about defaults, Spring Boot has its own opinions. If you are not specifying the details, it will use its own default configurations. If you want persistence, but don't specify anything else in your POM file, then Spring Boot configures Hibernate as a JPA provider with an HSQLDB database.

2- Introduction to Spring Boot

primary goals

- To provide a radically faster and widely accessible getting started development experience for all Spring development. Since spring community has evolved so big, it is time to re-invent the way how spring applications are deployed in much quicker turn around time.
- To be get started so quickely using the default values which are supported out of the box in the Spring Boot configurations.
- To provide bunch of non-functional features/solutions that are very much common to large scale projects (e.g. embedded servers, security, metrics, health checks, externalized configuration).

2- Introduction to Spring Boot

What Spring Boot brings to the table ?

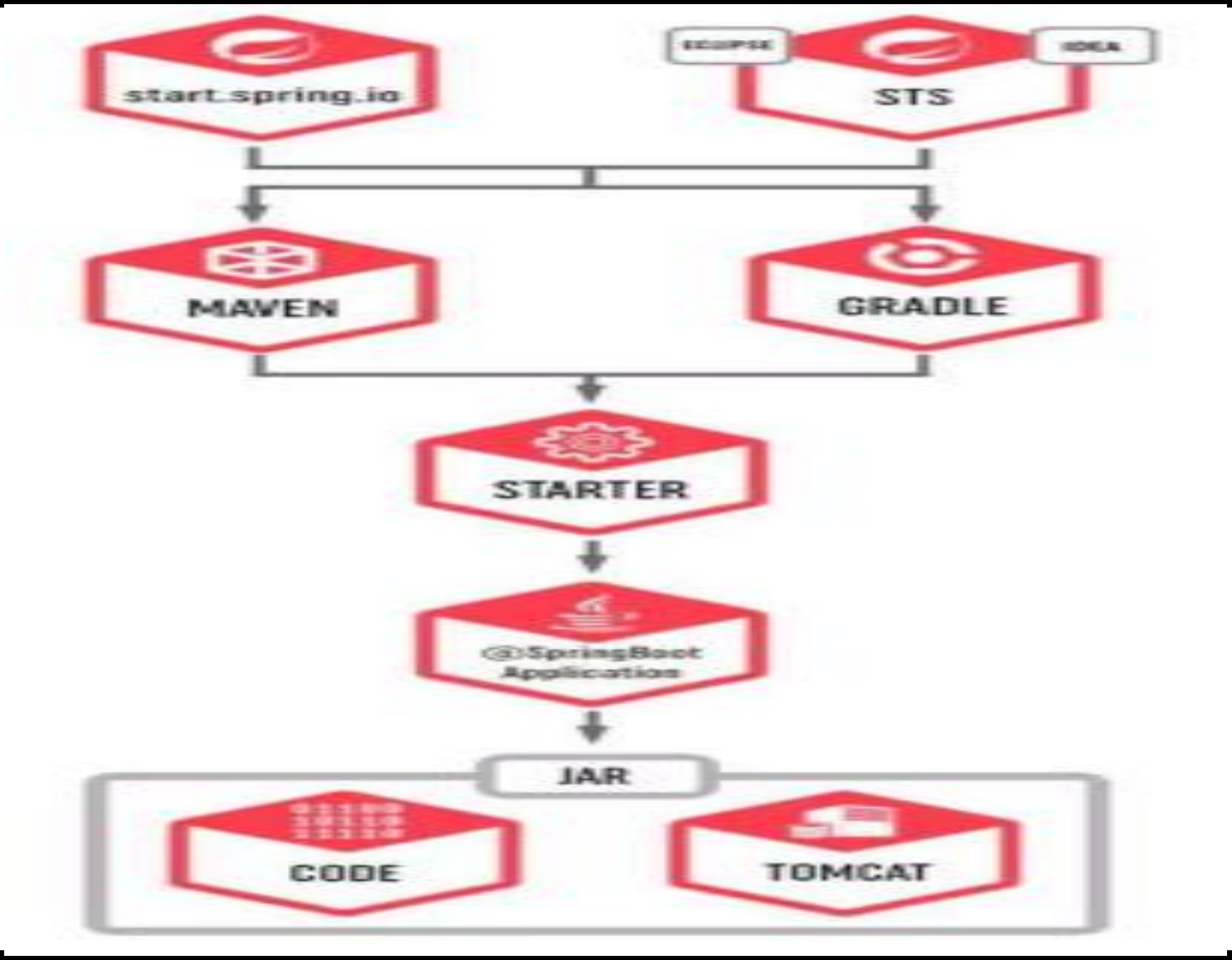
- Convention over configuration
- Standardization for Microservices
- Integrated Server for Development
- Cloud Support
- Adapt & Support for 3rd Party Library

2- Introduction to Spring Boot

How to use Spring Boot

- You can use spring initialize to create the initial setup. You can visit either start.spring.io or use STS (Spring Tool Suite) Support available in IDEA or Eclipse to choose all the Spring Boot Starters
- You need to also choose whether to use Maven or Gradle as the build tool.
- If you are using start.spring.io, you need to then download the zip and configure your workspace. Otherwise using your preferred IDE will automatically create the required file in the workspace.
- Add your code as required
- You can either use mvn clean package or use IDEA or Eclipse to build and create the jar file.
- By default the JAR would include integrated Tomcat server, so just by executing the JAR you should be able to use your program.

2- Introduction to Spring Boot



2- Introduction to Spring Boot

MyAppApplication Class

```
package com.apps;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MyAppApplication {

	public static void main(String[] args) {
		SpringApplication.run(MyAppApplication.class, args);
	}
}
```

2- Introduction to Spring Boot

@SpringBootApplication annotation

- @SpringBootApplication annotation
- Many Spring Boot developers always have their main class annotated with @Configuration, @EnableAutoConfiguration and @ComponentScan. Since these annotations are so frequently used together (especially if you follow the best practices above), Spring Boot provides a convenient @SpringBootApplication alternative.
- The @SpringBootApplication annotation is equivalent to using @Configuration, @EnableAutoConfiguration and @ComponentScan with their default attributes:

2- Introduction to Spring Boot

SpringApplication.run()

- You need to run Application.run() because this method starts whole Spring Framework. Code below integrates your main() with Spring Boot.
- public class SpringApplication extends Object
- Classes that can be used to bootstrap and launch a Spring application from a Java main method. By default class will perform the following steps to bootstrap your application:
  - Create an appropriate ApplicationContext instance (depending on your classpath)
  - Register a CommandLinePropertySource to expose command line arguments as Spring properties
  - Refresh the application context, loading all singleton beans
  - Trigger any CommandLineRunner beans
- In most circumstances the static run(Object, String[]) method can be called directly from your main method to bootstrap your application:

2- Introduction to Spring Boot

application.properties File

- spring.datasource.url=jdbc:mysql://localhost:3306/springboot
- spring.datasource.username=admin
- spring.datasource.password=admin
- spring.datasource.driver-class-name=com.mysql.jdbc.Driver
- # Allows Hibernate to generate SQL optimized for a particular DBMS
- spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL5Dialect
- # Number of ms to wait before throwing an exception if no connection is available.
- spring.datasource.tomcat.max-wait=10000
- # Maximum no of active conn that can be allocated from this pool at the same time.
- spring.datasource.tomcat.max-active=50
- # Validate the connection before borrowing it from the pool.
- spring.datasource.tomcat.test-on-borrow=true

2- Introduction to Spring Boot

```
# Keep the connection alive if idle for a long time (needed in production)
spring.datasource.dbcp.test-while-idle=true
spring.datasource.dbcp.validation-query=SELECT 1
# Naming strategy
spring.jpa.hibernate.naming.strategy=org.hibernate.cfg.ImprovedNamingStrategy
# Hibernate ddl auto (create, create-drop, update): with "update" the database
# schema will be automatically updated accordingly to java entities found in
# the project
spring.jpa.hibernate.ddl-auto=update

# Show or not log for each sql query
spring.jpa.show-sql = true
```

2- Introduction to Spring Boot

What is Spring Boot

The goal of Spring Boot is to provide a set of tools for quickly building Spring applications that are easy to configure.

The problem: Configuring Spring is hard

If you've ever written a Spring-based application, you know that a lot of work goes into configuring it just to get to "Hello, World." This isn't a bad thing: Spring is an elegant set of frameworks that require carefully coordinated configuration to work correctly. But that elegance comes at the cost of configuration complexity (and don't even get me started on all that XML).

The solution: Spring Boot

Enter Spring Boot. The Spring Boot website says it much more succinctly than I can: "Spring Boot makes it easy to create stand-alone, production-grade Spring based Applications that you can 'just run.' We take an opinionated view of the Spring platform and third-party libraries so you can get started with minimum fuss."

Basically, this means you can quickly get a Spring application up and running with very little configuration. What little configuration there is comes in the form of annotations, so no XML.

2- Introduction to Spring Boot

What is Spring Boot

All of that sounds great, right? But how exactly does Spring Boot work?

❑ It's opinionated

Spring Boot has opinions. This is just another way of saying that Spring Boot has reasonable defaults, so you can build an application quickly using these commonly used values.

As an example, Tomcat is a popular web container. By default, a Spring Boot web application uses an embedded Tomcat container.

❑ It's customizable

An opinionated framework isn't much good if you can't change its mind. You can easily customize a Spring Boot application to your liking, either in the initial configuration or later in the development cycle.

For example, if you prefer Maven, then you can easily make change(s) in your POM file to replace the Spring Boot default value. You'll do this later in the tutorial.

3- Features of Spring Boot

3- Features of Spring Boot

- If you want to create a Spring Boot project, you just need to use the online project generator:

https://start.spring.io/

3- Features of Spring Boot

- Spring is one of the most popular frameworks for building enterprise applications, but traditional Spring projects require heavy XML configuration, making them complex for beginners.

- Spring Boot solves this problem by providing a ready-to-use, production-grade framework on top of Spring. It eliminates boilerplate configuration, comes with an embedded server and focuses on rapid development.

3- Features of Spring Boot

Features of Spring Boot

- Spring Boot is built on top of the conventional Spring framework, providing all the features of Spring while being significantly easier to use. Here are its key features:

- **1. Auto-Configuration:** Automatically configures components (like Hibernate, JPA) based on dependencies. No need for manual XML setup.

3- Features of Spring Boot

Features of Spring Boot

- 2. **Easy Maintenance and Creation of REST Endpoints:** With annotations like @RestController, @GetMapping and @PostMapping, creating REST endpoints is straightforward

```
@RestController
@RequestMapping("/api")
public class MyController {
    @GetMapping("/hello")
    public String sayHello() {
        return "Hello, World!";
    }
}
```

3- Features of Spring Boot

Features of Spring Boot

- 3. **Embedded Tomcat Server:** Embedded Servers: Spring Boot includes an embedded Tomcat server, eliminating the need for manual setup. It also supports Jetty and Undertow, offering flexibility for different application needs.

- **4. Easy Deployment:** Spring Boot applications can be packaged as JAR or WAR files and deployed directly to servers or cloud environments. By 2025, it offers seamless integration with Docker and Kubernetes for easier cloud-native deployment and scaling.

- **5. Microservice-Based Architecture:** Spring Boot is well-suited for microservice architecture, where applications are split into independent, modular services. Unlike monolithic systems, this approach improves scalability, maintainability and deployment flexibility.

3- Features of Spring Boot

Major Spring Boot Versions

- Spring Boot 1.0 (April 2014)
- Spring Boot 2.0 (March 2018)
- Spring Boot 3.0 (November 2022) (Introduced Jakarta EE 9+, GraalVM support)
- Latest Version (2025): Spring Boot 3.x (Enhancements in observability, native images and containerization)

3- Features of Spring Boot

Applications of Spring Boot

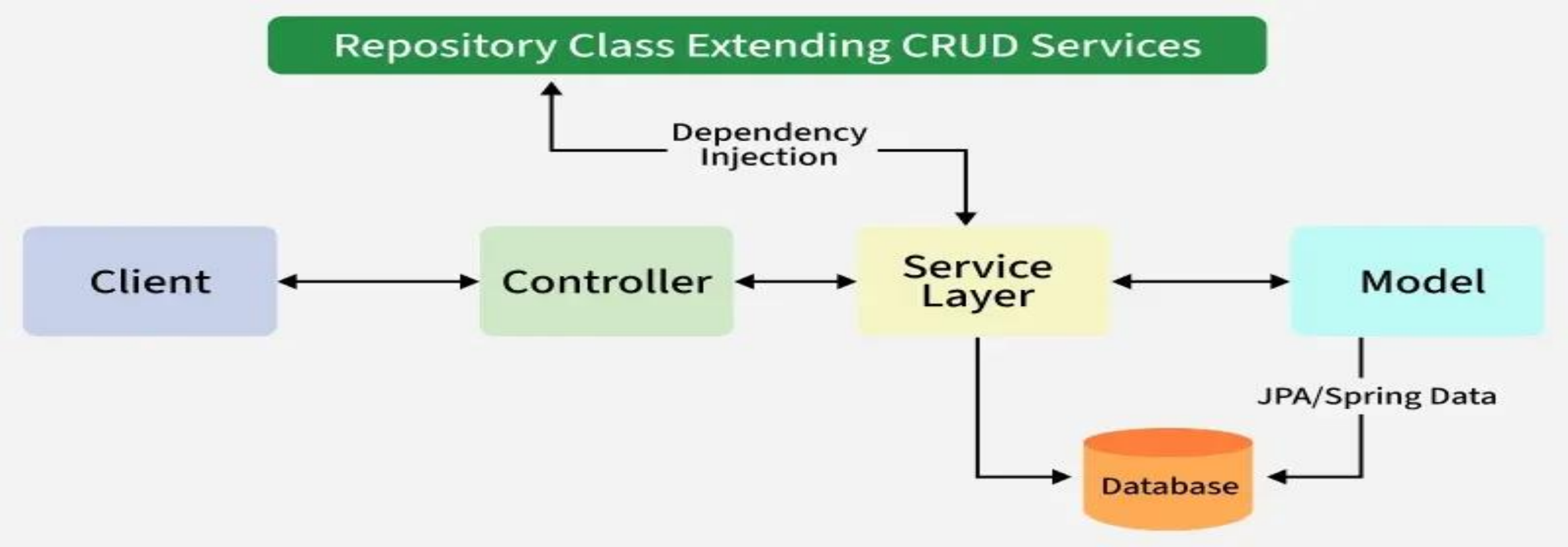
Spring Boot is widely used in modern software development because of its simplicity, scalability and production-ready features. Here are some of its major applications:

- 1. **Enterprise Applications:** Build complex applications such as Hospital Management Systems, Banking Systems or ERP solutions with minimal configuration.
- 2. **Cloud-Native Applications:** Seamless integration with Docker, Kubernetes and cloud platforms (AWS, Azure, GCP) for deployment and scaling.
- 3. **Real-Time Applications:** Supports Reactive Programming for chat applications, streaming platforms, IoT systems and event-driven systems.
- 4. **Batch Processing Applications:** With Spring Batch, Spring Boot is used for ETL (Extract, Transform, Load), report generation and large data processing.

4- Spring Boot Architecture

Spring Boot Architecture

To understand the architecture of Spring Boot, let's examine its different layers and components.



Spring Boot Architecture

To understand the architecture of Spring Boot, let's examine its different layers and components.

- **1. Client Layer**
  - This represents the external system or user that interacts with the application by sending HTTPS requests.

- **2. Controller Layer (Presentation Layer)**
  - Handles incoming HTTP requests from the client.
  - Processes the request and sends a response.
  - Delegates business logic processing to the Service Layer.

Spring Boot Architecture

- **3. Service Layer (Business Logic Layer)**
  - Contains business logic and service classes.
  - Communicates with the Repository Layer to fetch or update data.
  - Uses Dependency Injection to get required repository services.

Spring Boot Architecture

- 4. **Repository Layer (Data Access Layer)**
  - Handles CRUD (Create, Read, Update, Delete) operations on the database.
  - Extends Spring Data JPA or other persistence mechanisms.

- **5. Model Layer (Entity Layer)**
  - Represents database entities and domain models.
  - Maps to tables in the database using JPA/Spring Data.

- **6. Database Layer**
  - The actual database that stores application data.
  - Spring Boot interacts with it through JPA/Spring Data.

4- Spring Boot Architecture

Request Flow in Spring Boot

Client ->Controller ->Service ->Repository ->Database ->Response

- A **Client** makes an *HTTPS request* (GET/POST/PUT/DELETE).
- The *request* is handled by the **Controller**, which is mapped to the corresponding route.
- If *business logic* is required, the Controller calls the **Service Layer**.
- The Service Layer processes the logic and interacts with the **Repository Layer** to retrieve or modify *data* in the **Database.**
- The *data* is mapped using JPA with the corresponding Model/Entity class.
- The response is sent back to the **client.** If using Spring MVC with JSP, a JSP page may be returned as the response if no errors occur.

5- What is Maven ?

5- What is Maven ?

CREDITS

- www.geekforseeks.org
- Rochester Institute of Technology - Department of Software Engineering - Introduction to Software Engineering - REST Basics.pptx
- Sourabh Sharma, Modern API Development with Spring and Spring Boot, Packt Publishing Ltd., 2021

Time for some labs!